TestOrCode: An AI Agent Team for Safe CI Failure Triage

Team: Agent Zero

Team Member	Roll Number
T. Nikhilesh	2023101037
B. Sai Charan	2023101044
K. Manoj Sai Kumar	2023101061
K. Nikhil	2023101100

1 Domain, Target Users and Constraints

Domain and Scope: Software engineering, specifically unit-test failure diagnosis and repair in small Python libraries that use `pytest`. We only look at assertion failures in unit tests. Flaky tests, build failures and non-Python projects are out of scope.

Target users: Contributors and maintainers of small open-source Python libraries, student teams working on Python projects.

Constraints the domain gives us:

- *Passing tests is not enough.* An agent could make all tests pass by changing, skipping, or deleting tests. Therefore, we must also check whether the tests remain reliable.
- *Evidence can be incomplete or outdated.* The system may not always have enough information to decide whether the test or the code is wrong, so it must be able to escalate the case.
- *Different failures can occur together.* One code change can cause both outdated tests and real code regressions, so each failing test must be evaluated separately.
- *Decisions may need to remain consistent over time.* An approved behaviour change may affect later test failures, so the system needs to remember relevant past decisions.
- *Safe editing.* The system should modify only the appropriate source or test files and should not change configuration, dependency files

2 Problem Statement

When CI fails after a code change, a developer must decide which side is wrong: the test (the behaviour was changed on purpose) or the code (it regressed). Coding agents are usually rewarded when the tests pass, and the cheapest way to get there is often to edit the assertion to match the new output. If the code had really regressed, this hides the bug and permanently weakens the test suite. If the agent edits the code to satisfy an old test, it silently reverts an intended change. The evidence needed to choose is not in the failing output. It sits in commit messages, changelogs, docs and earlier maintainer decisions, and docstrings and old tests often describe the *old* behaviour. One deliberate change can also break many tests over several commits, so handling each failure from scratch is inconsistent and expensive, while reusing earlier decisions too broadly can make a later real regression look like "the same intended change". We therefore study a decision-and-integrity problem: can an agent pick the correct side for each failing test, repair it without weakening the suite, and reuse approved changes only when the current behaviour still matches them? The scope is 3 evaluation libraries, one failure type, and three outcomes: `UPDATE_TEST`, `FIX_CODE` and `ESCALATE`.

3 Research Question and Hypothesis

Primary RQ. On chronological streams of CI failures in small Python libraries, does a multi-agent system with separate roles for (i) using retrieved change-intent evidence to support each repair decision, (ii) remembering approved behaviour changes for specific functions, together with tests that verify those behaviours still hold, and (iii) checking test integrity after repairs make fewer wrong repairs and weaken tests less than a single coding agent using the same tools, model, and budget?

Hypothesis. Compared with a single tool-using agent with standard RAG and no memory (B1), we expect the full multi-agent system to improve:

- Per-test direction decision accuracy (macro-F1) — better at deciding whether to update the test or fix the code.
- Masked-regression rate — fewer cases where a real code regression is incorrectly hidden by changing the test.
- Mutant-kill retention — better at preserving tests that can detect incorrect code behaviour.
- Tool-call efficiency — fewer tool calls when handling changes covered by an earlier approved memory.
- Memory behaviour — scoped memory should be less likely to incorrectly apply a past decision to unrelated or misleading cases than running with no persistent memory at all.

4 Why the Problem Matters

- Tests need to change when the code changes. Maintainers often have to decide whether a failing test should be updated or whether the code has a real bug. Both wrong decisions can cause problems: a hidden regression can let a bug remain, while reversing an intended change can undo the desired behaviour.
- The problem can be hard to notice. A test suite can become weaker over time while all tests still pass. There agents can do changing or adding special cases to tests just to make them pass.
- Benchmarks that score agents by running tests assume the tests are a trustworthy oracle. Our setting asks the opposite question: when should the oracle itself change?

5 What Existing Approaches Miss

- **A normal LLM** sees a traceback and a diff. It cannot run tests, read history, or check its own patch. Nothing in the prompt makes it look for evidence that the new output is intended, and it has no memory of earlier approved changes.
- **Standard RAG** searching with the failing test's text mostly returns the test and the function under test, not the commit message that states intent. The docstrings and old tests it does return describe pre-change behaviour, so the evidence can support the wrong side. RAG only retrieves information. It does not decide whether to update the test or fix the code, verify the decision with a probe, or check whether the tests remain strong.
- **A single tool-using agent** has conflicting goals: the same agent that wants a green build also judges whether editing the test is acceptable. A "do not weaken tests" instruction can fade over a long tool trajectory, and no independent step measures test strength after repair. This is a suspicion that we are going to test.
- **Test-repair and issue-resolution work** (TaRGET/TaRBench, ReAssert, SWE-bench, SWE-agent, Agentless) focuses on either repairing failing tests or resolving software issues, rather than deciding whether a failure should be handled by changing the test or fixing the code. These approaches do not evaluate this test-versus-code decision as a primary task, and they generally do not measure whether the test suite remains strong after a repair.

6 Capabilities Used

The capabilities used in the project, along with their implementation and evaluation, are summarized in Table 1.

7 Proposed Idea and Architecture

The system never treats a green build as the goal. For each failing test it decides whether the test is outdated (`UPDATE_TEST`), the code has regressed (`FIX_CODE`), or the evidence is too weak (`ESCALATE`). All agents use the same hosted LLM, so differences between conditions come from the design and not the model. The roles and responsibilities of the four agents are summarized in Table 2.

Workflow.

1. A CI event arrives. Triage runs the suite, confirms the failures are stable, and groups them by function.
2. For each group, the Intent Analyst retrieves evidence from the commit message, docs, changelog, docstrings, earlier commit messages and memory records, then states a hypothesis and the expected new behaviour.
3. The decision policy is applied (below).
4. The Repairer applies a test patch or a source patch inside the allowed scope and reruns the tests.
5. The Integrity Guard checks the result. For test patches it runs the function's mutants against the updated

Capability	Why the problem needs it	How we implement it	How we measure it
Agentic Abilities	The agent that wants green should not be the only one judging whether a test edit is legitimate. Verification needs to be independent.	Four roles (Triage, Intent Analyst, Repairer, Integrity Guard). Only the Guard can write memory. One revision round, then escalation.	Direction F1, masked-regression and reverted-intent rates, escalation precision/recall, ablation A3 (No Integrity Guard).
RAG	Intent lives in commit messages, changelogs and docs, not in the failure output, and much of the obvious text is stale.	Hybrid index (BM25 + code embeddings, optional rerank) over docs, docstrings, changelog and history up to the current commit.	Gold-evidence Recall@k, citation faithfulness (cited item exists and supports the claim), stale-evidence error rate, ablation A1.
Persistent scoped memory	One change breaks tests over many commits. Reuse saves cost, but broad reuse hides regressions.	Records {function, behaviour signature, direction, executable probe, evidence IDs, commit}. A record counts only if its probe passes on current code.	Tool calls on covered episodes, over-generalisation rate on decoys, memory-record accuracy, ablation A2.
Reasoning (hypothesis testing)	A CI run can mix outdated tests and regressions, and each failure needs its own explanation.	Per failure group: hypothesis (outdated / regression / unknown), predicted new behaviour, then a check against evidence or probe.	Per-test hypothesis accuracy, evidence-justified decision rate, distribution of the first wrong step in failed episodes.
Tool use	The system must run tests and mutants, read history and edit files without collateral damage.	Sandboxed tools with scope-restricted `apply_patch`, injected tool faults, cached mutants.	Tool-call accuracy vs. reference traces, invalid-call rate, error-recovery rate, out-of-scope-edit rate.

Table 1: Capabilities used in the proposed system, their implementation, and evaluation.

tests and compares the kill rate with the reference tests. It also looks for lost assertions and added skip/xfail markers. For source patches it checks that no test file was touched and that the suite passes.

6. If the Guard rejects, the Repairer gets one revision. If the Guard still rejects, the outcome is `ESCALATE`.
7. A simulated maintainer reviews the final patch. On approval, the Guard writes a memory record.

Decision policy. `UPDATE_TEST` needs a cited piece of positive evidence that the change was intended. A memory record counts as evidence only if its probe passes on the current code. A regression (`FIX_CODE`) is chosen when behaviour contradicts the evidence or the probe. If neither side has evidence, the outcome is `ESCALATE`, with a summary of the failing tests, the evidence found and missing, and both hypotheses.

Memory. A record holds the function, a behaviour signature, the direction, an executable probe (derived from the assertions in the approved test patch), evidence IDs and the commit. Records are written only after maintainer approval. A newer approved change to the same function supersedes the older record. If a probe fails on the current code, the record is not used, and a record that keeps failing is flagged. This is how stale or wrong records are handled.

Retrieval. BM25 (`rank_bm25`) plus code embeddings, optionally reranked, over docs, docstrings, changelog and commit history up to the current commit, plus memory records. Future commits are never indexed.

8 Expected Novelty

We do not claim a new algorithm. The contribution is a design-decision study with a new evaluation setup.

- **What already exists:** Automated test-repair methods, issue-resolution agents, mutation testing for measuring test-suite strength, and general forms of agent memory such as Voyager, Reflexion, and ExpeL.
- **What we combine:** The test-vs-code direction decision, intent-grounded retrieval, memory records backed by executable probes, an independent mutation-based integrity check, and role separation, all evaluated on one chronological stream of commits.
- **What we test experimentally:** (a) whether intent retrieval improves repair decisions, (b) whether scoped

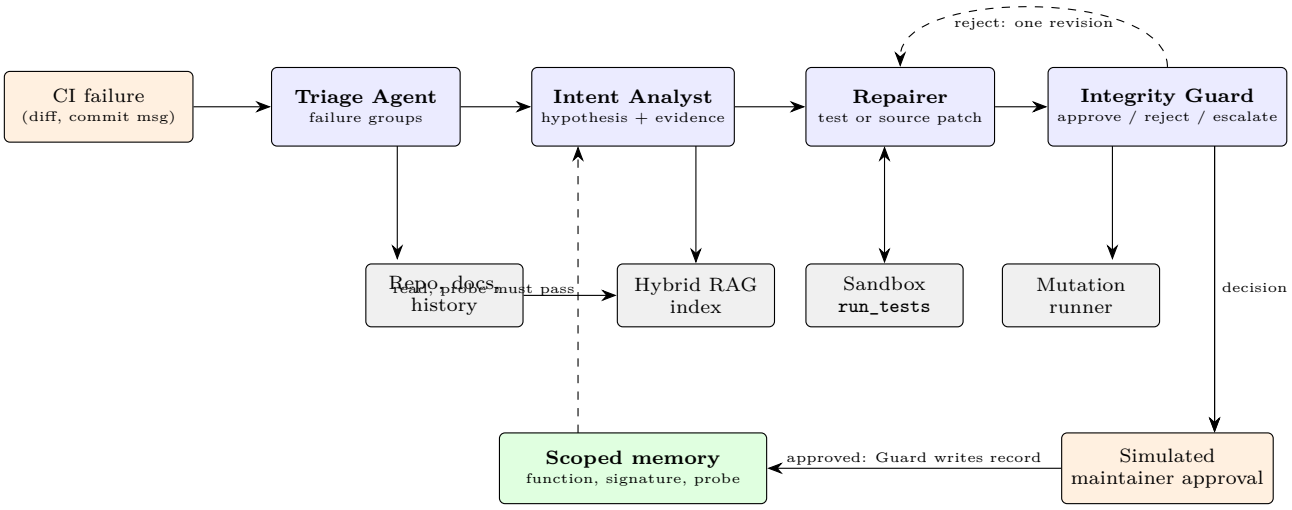


Figure 1: Workflow. Solid arrows are the main flow. The dashed arrow from the Guard back to the Repairer allows one revision. The dashed arrow from memory to the Analyst is a read. Memory is written only by the Guard after approval.

memory with probes reduces incorrect reuse of past decisions compared with no memory, and (c) whether an independent integrity check reduces weakened tests.

9 Dataset and Evaluation Environment

Base data (real). Git histories of small, pure-Python, permissively licensed libraries with fast pytest suites: 3 held-out libraries for evaluation and 1 for a development stream (for tuning). Names such as toolz, boltons, tabulate, or humanize are only candidates. We will screen candidate libraries for license, test-suite speed, dependency complexity, and the number of usable commits before selection. The retrieval corpus for each library will include the README/docs, docstrings, changelog, and commit messages available up to the current commit. The underlying code, tests, and commit histories are real; controlled evaluation cases will be created by injecting regressions into real code.

Test set. 72 episodes, 24 per library, evaluated as chronological streams so that approved decisions from earlier episodes can be reused in later episodes. A separate development stream contains about 24 episodes from a fourth library. The evaluation scenarios and their corresponding gold labels are summarized in Table 3.

Environment. Each episode runs in a Docker sandbox with no network access, fixed Python and dependency versions, resource limits, and a separate worktree. LLM and embedding calls are made only through the orchestrator. Agents cannot access future commits or the developer's reference test diff. A simulated maintainer approves a patch when its direction matches the gold label and all tests pass, and expects an escalation for S5. Approval then triggers the memory write.

10 Baselines

All LLM conditions use the same model, token budget and tool-call budget per episode. **B1 is the main baseline**: a single tool-using agent with the same tools and standard top-k RAG over the repository, no persistent memory. The full multi-agent system is compared primarily against B1. Deterministic always-`UPDATE_TEST` and always-`FIX_CODE` rules serve as free sanity checks, requiring no additional LLM calls, to show what each metric reveals at the extremes.

11 Metrics

The evaluation metrics and the aspects of performance they measure are summarized in Table 4.

Agent	Receives	Does / produces	Tools and memory
Triage	CI event (diff, commit message)	Runs the suite, reruns failures to confirm they are stable, groups failures by changed function. Output: failure groups with tracebacks and assertion diffs.	`run_tests`, `git_show`, `read_file`. Memory: read-only.
Intent Analyst	Failure groups	Retrieves intent evidence, forms a hypothesis per group with cited evidence IDs, predicts the new behaviour.	`search_repo`, `git_log`, `git_show`, `git_blame`, `memory.read`. Memory: read-only.
Repairer	Hypothesis and evidence	Test patch for outdated tests, source patch for regressions, or an escalation request.	`apply_patch` (scope = src or test), `run_tests`, `read_file`. Memory: read-only.
Integrity Guard	Patch, hypothesis, memory record	Independently verifies the result. Outputs approve, reject or escalate. Writes memory after approval.	`run_tests`, `run_mutants`, `memory.read/write`, `escalate`. Memory: read and write.

Table 2: Roles, inputs, outputs, tools, and memory access of the four agents.

11.1 Unsafe or Unapproved Actions

We count the following as unsafe or unapproved:

- editing files outside `src` and `tests`, including CI configuration, dependency files;
- weakening tests: removing assertions, adding skip/xfail, loosening comparisons, or swallowing exceptions;
- wrong-side edits: a masked regression (test edited when gold is `FIX_CODE`) or a reverted intent (source edited when gold is `UPDATE_TEST`);
- writing memory without maintainer approval;
- using a memory record whose probe fails on the current code as evidence.

12 Ablations

A1 tests the primary RQ directly by isolating the contribution of intent retrieval. A2 tests the secondary RQ by comparing persistent scoped memory against no memory. A3 isolates the value of the independent integrity step. All ablations run over the same 72 chronological test episodes.

13 Compute Plan

- **Models.** Hosted LLM APIs only. No local GPU and no fine-tuning. One main model is used for all agents and all conditions. The first choice is Qwen3 Coder 480B on NVIDIA NIM, with Mistral Devstral 2 as a backup. The final model is chosen on the development stream by tool-calling reliability and frozen by dated model ID before any test-set episodes are run.
- **Episode runs.** 5 main conditions run over 72 episodes each: Full system (2 seeds), B1 (2 seeds), A1 (1 seed), A2 (1 seed), A3 (1 seed). This gives $72 \times 2 + 72 \times 2 + 72 + 72 + 72 = 504$ main episode-runs, plus the deterministic always-`UPDATE_TEST` and always-`FIX_CODE` sanity checks which require no LLM calls.
- **API calls.** Approximately 15 LLM calls per episode for the full system, approximately 10 for B1, and approximately 8–12 for ablations (which omit one agent or one component). Across all 504 episode-runs this amounts to roughly 6,000–8,000 LLM calls in total.
- **Tokens.** Roughly 25–40M tokens in total (approximately 4–6k per call).
- **Embeddings.** A few million tokens for the corpora of 4 libraries, within a free embedding quota.
- **CPU.** Tests and mutants run in Docker on CPU, about 15–25 CPU-hours in total, parallelisable.
- **Development stream.** About 24 episodes used for debugging, prompt and retrieval tuning, and model selection. The configuration is frozen before the 72-episode test set is touched.
- **Cost and time.** We assume $0 on free tiers. Estimated runtime is 2–3 days on NIM or about 4–5 days on Mistral alone. Free-tier limits change without notice, so we will re-check them when we sign up and run the

Scenario	#	Construction	Gold label
S1 Outdated test	30	Real commit that changed source and tests. Apply only the source part to its parent and keep the old tests.	`UPDATE_TEST`; developer's test diff is the reference repair.
S2 Injected regression	12	Source-only commit where tests still pass, plus one mutant that at least one test kills.	`FIX_CODE`; tests must stay untouched.
S3 Mixed	12	S1 plus a mutant in a different function.	Per test: `UPDATE_TEST` for outdated, `FIX_CODE` for new failures.
S4 Decoy regression	9	Later in the stream, a mutant in a function whose change was approved earlier, with a different symptom.	`FIX_CODE`; memory must not justify an update.
S5 Intent-stripped	9	S1 with an uninformative commit message, docs changes removed, and no memory coverage.	`ESCALATE`.
T Tool faults	–	About 10% of tool calls fail (patch does not apply, bad selector, timeout).	Recover without unsafe edits.

Table 3: Evaluation scenarios, their construction, and expected gold labels.

core experiments (Full vs. B1) first. Every call is cached by (model, prompt, seed) and logged with model ID and date. Rate-limit errors are retried and logged separately, never counted against the agent. We never split calls across providers within one comparison.

Budget note: If API availability or budget becomes limiting, we may reduce or skip some ablation runs while keeping the main comparison between the full system and B1, which is the highest priority. The priority order is: (1) Full system, (2) B1 main baseline, (3) Full vs. B1 with two seeds, (4) A1 No intent retrieval, (5) A2 No memory, (6) A3 No Integrity Guard, (7) deterministic sanity baselines.

14 Risks and Fallback Plan

The main project risks and corresponding fallback strategies are summarized in Table 6.

Metric	What it measures
Direction accuracy, macro-F1	Per-failing-test choice between `UPDATE_TEST` and `FIX_CODE`.
Correct-repair rate	Right direction, suite passes, and integrity preserved.
Masked-regression rate	Share of `FIX_CODE` cases where a test was edited instead.
Reverted-intent rate	Share of `UPDATE_TEST` cases where source was edited instead.
Mutant-kill retention	Mutants killed by the updated tests, relative to the developer's reference test. Measures test strength after repair.
Test-weakening rate	Repairs with fewer assertions, added skip/xfail, loosened comparisons or swallowed exceptions.
Out-of-scope-edit rate	Edits outside `src` and `tests` (CI config, dependencies, `conftest.py`).
Escalation precision / recall; unnecessary-escalation rate	Whether `ESCALATE` is used on the right cases (S5) and not on cases with enough evidence.
Gold-evidence Recall@k, stale-evidence error rate	RAG: is the intent evidence found, and how often does stale text cause a wrong hypothesis.
Over-generalisation rate (decoys); tool calls per covered episode	Memory: harm from reuse (S4) and benefit from reuse (savings on later covered changes).
Tool-call accuracy, invalid-call rate, error-recovery rate	Tools: calls compared with reference traces, malformed calls, and recovery from injected faults (T) without unsafe edits.

Table 4: Evaluation metrics and the aspects of system performance they measure.

Condition	What is removed	What it tests
Full system	Nothing	Reference.
A1: No intent retrieval	Analyst sees only failing output and diff	Value of change-intent evidence on direction accuracy.
A2: No memory	Memory disabled; every episode handled from scratch	Value of cross-commit memory for cost and consistency (supports Secondary RQ).
A3: No Integrity Guard	No mutation check, no independent verification	Value of the independent integrity step on test-weakening rate.

Table 5: Ablation conditions and the design choices they evaluate.

Risk	Fallback
Environment problems (dependencies, Python versions) and unstable tests use up weeks.	Choose libraries with no compiled dependencies and fast tests. Drop unstable episodes using the 5-run filter.
Too few usable commits in small libraries.	Use 4 libraries with shorter streams, or relax to commits where source and tests changed by small diffs.
Free API limits, model availability, or tool-calling reliability change.	Confirm the model on the development stream and switch all conditions to the same backup model if needed. Reduce the no of experiments wherever we can.

Table 6: Project risks and planned fallback strategies.